

OS LAB WEEK – 3

3) Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>
#include <string.h>

#define MAX 10

struct Process {
    int pid, arrival, burst, completion, turnaround, waiting;
    char type[10];
};

int main() {
    int n;
    struct Process SQ[MAX], UQ[MAX];
    int sq = 0, uq = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        struct Process p;
        printf("\nPID: ");      scanf("%d", &p.pid);
        printf("Arrival Time: ");  scanf("%d", &p.arrival);
        printf("Burst Time: ");    scanf("%d", &p.burst);
        printf("Type (System/User): "); scanf("%s", p.type);

        if (strcmp(p.type, "System") == 0)
            SQ[sq++] = p;
        else
            UQ[uq++] = p;
    }

    int current_time = 0;
    int sq_front = 0, uq_front = 0;
    struct Process result[MAX];
    int r = 0;

    while (sq_front < sq || uq_front < uq) {

        while (sq_front < sq) {
            struct Process *P = &SQ[sq_front];
```

```

        if (current_time < P->arrival)
            current_time = P->arrival;
        current_time += P->burst;
        P->completion = current_time;
        P->turnaround = P->completion - P->arrival;
        P->waiting = P->turnaround - P->burst;
        result[r++] = *P;
        sq_front++;
    }

    while (sq_front >= sq && uq_front < uq) {
        struct Process *P = &UQ[uq_front];
        if (current_time < P->arrival)
            current_time = P->arrival;
        current_time += P->burst;
        P->completion = current_time;
        P->turnaround = P->completion - P->arrival;
        P->waiting = P->turnaround - P->burst;
        result[r++] = *P;
        uq_front++;
    }
}

printf("\n%-5s %-8s %-8s %-8s %-12s %-12s %-8s\n",
        "PID", "Type", "AT", "BT", "CT", "TAT", "WT");
printf("-----\n");

float total_tat = 0, total_wt = 0;

for (int i = 0; i < r; i++) {
    printf("%-5d %-8s %-8d %-8d %-12d %-12d %-8d\n",
            result[i].pid, result[i].type,
            result[i].arrival, result[i].burst,
            result[i].completion, result[i].turnaround, result[i].waiting);
    total_tat += result[i].turnaround;
    total_wt += result[i].waiting;
}

printf("-----\n");
printf("Average TAT : %.2f\n", total_tat / n);
printf("Average WT : %.2f\n", total_wt / n);

return 0;
}

```

Enter number of processes: 2

PID: 1

Arrival Time: 0

Burst Time: 4

Type (System/User): System

PID: 2

Arrival Time: 3

Burst Time: 6

Type (System/User): User

PID	Type	AT	BT	CT	TAT	WT
1	System	0	4	4	4	0
2	User	3	6	10	7	1

Average TAT : 5.50

Average WT : 0.50

=== Code Execution Successful ===